Reproducing the NJIT CCSC Deep-Learning Tools

Xiangsheng (Harry) Bao, University of Sydney, October 2026

1. Executive Summary

The request was: "For each tool, click the Citation button, download the tool to reproduce all results and let me know how the tool works." This report covers the 19 tools listed on https://nature.njit.edu/solardb/ccsc, which I checked against the live page, each at the software version its Citation button points to. I also include MagNet, a related SolarDB tool.

I followed all 19 Citation buttons on the CCSC page and the one on the CCDC page. Each button opens a Zenodo record. Zenodo at first timed out from my environment (HTTP 504), so I resolved each DOI through DataCite, the registry Zenodo deposits to, and checked the archives directly once Zenodo became reachable. For 13 tools the cited record is a GitHub release tag (for example LSTM-flare v1.0.2 and SYMHnet v1.0), and I ran or diffed that tagged version. For the other 6 the record is a direct Zenodo upload, and I compared each archive file by file with the GitHub version I ran (§3.5). Four match in content. MIST's archived test.py fails as cited (this is fixed on GitHub), and SolarCNN's record is restricted-access.

I built four isolated Python environments on a 4-core CPU machine without a GPU. For each tool I ran the authors' released test or prediction path with their pretrained models and compared the outputs with the authors' stored outputs and with the tables in the published papers. I had 16 full texts, 14 from arXiv plus the *Scientific Reports* and *Frontiers* papers. Where a released evaluation looked questionable, I added leakage-aware checks: chronological hold-outs, grouped splits, persistence baselines and evaluation without label post-processing.

In the table, "tool level" means that the released model and data reproduce the authors' stored outputs, and "paper level" means that a headline number from the paper is reproduced.

Status	Tools
Reproduced at tool and paper level	SINet (F10.7 RMSE 3.88 and 16.32 sfu at 1 and 60 days, as in the paper); DeepGeoCME (TSS 0.757, as in the paper)
Reproduced at tool level; the paper's own evaluation cannot be run from the release	SEP-prediction (12/12 stored rows exact; the paper reports cross-validation means over data and folds that are not shipped); SDNN (all 5 maps match, r = 1.000000; the paper's Milne–Eddington reference maps are not shipped); FibrilNet (masks agree with the stored ones exactly as well as two of my own MC-dropout runs agree with each other; the paper's reference skeletons are not shipped); SolarUnet (masks byte-identical; the paper's statistics use 147 frames, only 3 are shipped); Kp-prediction (within MC noise; paper behind a paywall)
Partially reproduced	SolarCNN: stored predictions reproduced to 3×10^{-5} G. With the training normalisation that the shipped test.py omits, SSIM rises from 0.812 to 0.892 (paper: 0.833 to 0.904), but only on the 3 shipped samples of the paper's 76. LSTM-flare-prediction, RNN-CME-prediction, SolarFlareNet, SYMHnet, MIST, FlareML, TSInet and CMETNet: stored outputs reproduced, but the paper's numbers are not reached or the released evaluation differs from the paper's (§5, §6)
Attempted but unsuccessful	Dst-prediction: stored outputs reproduced, but the released `predict()` replaces most predictions with 0.9 × the true label. The raw model's R^2 (0.06–0.38) is far below the paper's (about 0.6–0.75, read off its Figure 3)
Blocked by missing data or compute	DeepHMI: the 1,000-step diffusion takes about 37 CPU-hours, so I re-scored the authors' output instead. MagNet: the package on nature.njit.edu lacks `layers/attention.py`, which its model file imports
Not attempted	LLMFlare: needs gated Llama-3.1-8B weights, a paid Gemini API key and a GPU, and I did not incur costs without authorisation

Main findings

Validation design had the largest effect on the scores. The pretrained MIST transformer reaches TSS 0.906 on its released random split of overlapping 24-hour windows. When I compared its test windows with the

authors' own training array, which is in the Zenodo archive, 95.8 % of them shared 23 of their 24 hours with a training window. On 2015–2018 data that is independent of the training data both in time and in active regions, MIST reaches TSS 0.197, and its AUC falls from 0.997 to 0.833. On that same test set, the 2019 ≥M LSTM retrained on its published split reaches TSS 0.73–0.75.

The 2019 LSTM data split is exemplary. It is chronological (2010–13 / 2014 / 2015–18) with zero shared active regions, so I used it to measure how much choosing the decision threshold on the test set inflates TSS. Compared with choosing the threshold on the 2014 validation year, the inflation is +0.005 for ≥C, +0.009 for ≥M and +0.052 for ≥M5.0. Compared with the tool's fixed threshold, it is +0.009, +0.018 and +0.143. Rare classes are the most sensitive.

In several tools the released models and data do not match the paper, or the released evaluation does not measure what it appears to measure. RNN-CME's released pretrained models score TSS 0.26–0.49 on the paper's own test set, against 0.45–0.60 in its Table 4. The two copies of the data in the release also store the 18 SHARP columns in different orders, and the code's feature ranking matches the paper's Table 2 only under the physically implausible order. For SolarFlareNet, 8 of the 9 shipped models have no skill on the bundled test data (paper TSS 0.71–0.84). Dst-prediction uses label-dependent post-processing, CMETNet chooses the best of 10 models on the test year, and FlareML adjusts its metrics on the display side and scales the test data per batch. SolarCNN's test.py omits the training normalisation, so the shipped predictions show no improvement although the model does. SYMHnet's 1 h and 2 h data are identical and its target is the next row. The cited v1.0 code reproduces the paper's per-storm RMSE for 3 of 7 test storms (within about 2 %), so the paper's numbers appear to come from this pipeline.

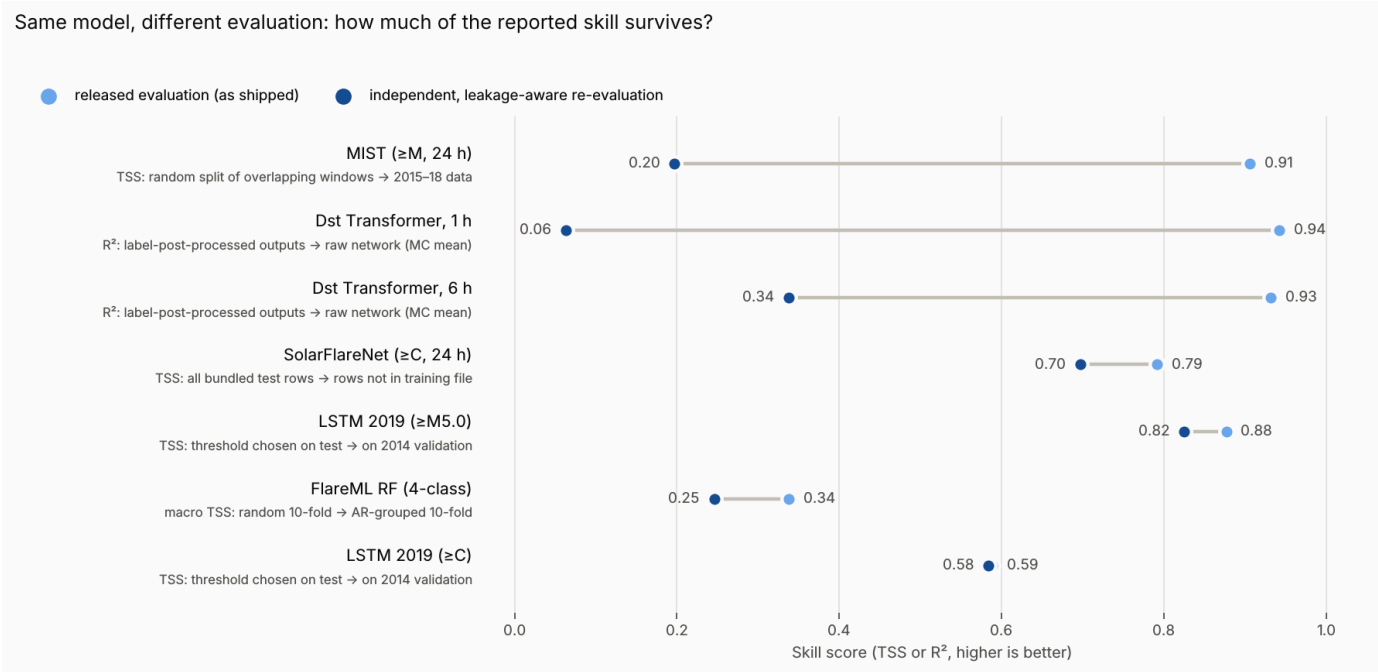


Figure 1. Each model scored two ways. Light dots show the evaluation shipped in the repository, dark dots an independent, leakage-aware re-evaluation of the same weights or code. The numbers are in §5.

These are observations about the released software at the cited versions. Where a paper's own evaluation data are not in the release, I say so rather than infer anything about the paper. I would value the authors' guidance wherever the release and the paper appear to differ.

2. Tools and Publications

Each tool's Citation button points to a Zenodo software archive. The table gives that archive and the tool's paper, in the order of the CCSC page. The full APA and BibTeX records are in `papers/citations/README.md`, and the full 16-field record for each tool is in `tool_inventory.md`.

#	Tool	Publication	Paper DOI / arXiv	Zenodo DOI (version)
1	SolarUnet	Jiang et al., *Identifying and Tracking Solar Magnetic Flux Elements with Deep Learning*, ApJS 250:5 (2020)	10.3847/1538-4365/aba4aa; arXiv:2008.12080	10.5281/zenodo.5643647 (v.1.2)
2	FlareML	Abduallah et al., *DeepSun: Machine-Learning-as-a-Service for Solar Flare Prediction*, RAA 21:160 (2021)	10.1088/1674-4527/21/7/160; arXiv:2009.04238	10.5281/zenodo.7506997 (v1.5)
3	RNN-CME-prediction	Liu, Liu, Wang & Wang, *Predicting CMEs Using SDO/HMI Vector Magnetic Data Products and RNNs*, ApJ 890:12 (2020)	10.3847/1538-4357/ab6850; arXiv:2002.10953	10.5281/zenodo.7415731 (v1.3)
4	LSTM-flare-prediction	Liu, Liu, Wang & Wang, *Predicting Solar Flares Using a Long Short-term Memory Network*, ApJ 877:121 (2019)	10.3847/1538-4357/ab1b3c; arXiv:1905.07095	10.5281/zenodo.5739321 (v1.0.2)
5	SEP-prediction	Abduallah et al., *Predicting SEPs Using SDO/HMI Vector Magnetic Data Products and a BiLSTM Network*, ApJS 260:16 (2022)	10.3847/1538-4365/ac5f56; arXiv:2203.14393	10.5281/zenodo.7516609 (v1.4)
6	Dst-prediction	Abduallah et al., *Forecasting the Disturbance Storm Time Index with Bayesian Deep Learning*, FLAIRS-35 (2022)	10.32473/flairs.v35i.130564; arXiv:2205.02447	10.5281/zenodo.7529705 (v1.1)
7	TSInet	Abduallah et al., *Reconstruction of Total Solar Irradiance by Deep Learning*, FLAIRS-34 (2021)	10.32473/flairs.v34i1.128356; arXiv:2107.11042	10.5281/zenodo.7516668 (v1.6)
8	Kp-prediction	Abduallah, Wang, Xu & Wang, *A Transformer-Based Framework for Geomagnetic Activity Prediction*, ISMIS 2022, LNCS 13515	10.1007/978-3-031-16564-1_31	10.5281/zenodo.7514451 (v1.1)
9	CMETNet	Alobaid et al., *Predicting CME Arrival Time through Data Integration and Ensemble Learning*, Front. Astron. Space Sci. 9:1013345 (2022)	10.3389/fspas.2022.1013345	10.5281/zenodo.7516684 (v.1.1)
10	FibrilNet	Jiang et al., *Tracing Hα Fibrils through Bayesian Deep Learning*, ApJS 256:20 (2021)	10.3847/1538-4365/ac14b7; arXiv:2107.07886	10.5281/zenodo.7521568 (v1.6)
11	SDNN	Jiang et al., *Inferring LOS Velocities and Doppler Widths from Stokes Profiles of GST/NIRIS Using Stacked DNNs*, ApJ 939:66 (2022)	10.3847/1538-4357/ac927e; arXiv:2210.04122	10.5281/zenodo.7521580 (v1.3)
12	SolarFlareNet	Abduallah, Wang, Wang & Xu, *Operational Prediction of Solar Flares Using a Transformer-Based Framework*, Sci. Rep. 13:13665 (2023)	10.1038/s41598-023-40884-1	10.5281/zenodo.10201356 (tag v1.3)
13	SYMHnet	Abduallah et al., *Prediction of the SYM-H Index Using a Bayesian Deep Learning Method with UQ*, Space Weather 22, e2023SW003824 (2024)	10.1029/2023SW003824; arXiv:2402.17196	10.5281/zenodo.10602518 (tag v1.0)
14	SolarCNN	Xu et al., *Super-Resolution of SOHO/MDI Magnetograms ... Attention-Aided CNN*, Sol. Phys. 299:36 (2024)	10.1007/s11207-024-02283-1; arXiv:2403.18302	10.5281/zenodo.15250172 (direct upload)

#	Tool	Publication	Paper DOI / arXiv	Zenodo DOI (version)
15	MIST	Software record: Chaudhary, Abduallah & Wang (2025), *A New Deep Learning Method for Solar Flare Prediction*. Closest paper I found: Chaudhary, Wang, Xu, Xu & Zhang, *Prediction of Solar Flares Using Photospheric Magnetic Field Parameters with Deep Learning* (2026); the link to MIST is my inference (§5.1)	arXiv:2606.21896	10.5281/zenodo.16780646 (direct upload)
16	DeepHMI	Xu, Xu, Wang, Li & Wang, *Improving the Spatial Resolution of SDO/HMI … Using GST/NIRIS Data*, A&A (2025)	10.1051/0004-6361/202453581	10.5281/zenodo.17059248 (direct upload)
17	SINet	Wang Z. et al., *Daily Predictions of F10.7 and F30 Solar Indices with Deep Learning*, JGR Space Physics (2026)	10.1029/2025JA034868; arXiv:2604.10045	10.5281/zenodo.20274899 (direct upload)
18	LLMFlare	Software record: Chaudhary, Abduallah & Wang (2026), *Explainable Solar Flare Prediction Using Large Language Models*. No paper found	none	10.5281/zenodo.20738748 (direct upload)
19	DeepGeoCME	Yan, Wang, Wang, Lee, Jing, Xu, Xu & Yurchyshyn, *Deep Learning-Enabled Prediction of Geoeffective CMEs Using SOHO and SDO Observations* (2026)	arXiv:2605.24748	10.5281/zenodo.22955090 (direct upload, 1.0.0)
–	MagNet (SolarDB, not on the CCSC page)	Jiang et al., Sol. Phys. 298:87 (2023); Yesmanchyk et al., *Out-of-Sample Validation of MagNet* (2026)	10.1007/s11207-023-02180-z; arXiv:2211.02278; arXiv:2601.15926	none

3. Methodology

3.1 Identifying the tools and following the Citation buttons

The CCSC page (nature.njit.edu/solardb/ccsc) builds its 19 tiles with a script, `js/script.js`, from a data file, `ccsc_pages/pages_data.json`. Each tile title links to `github.com/ccsc-tools/<repo>`, and each Citation button links to a Zenodo record. I saved both files and an HTML snapshot of the page in `papers/citations/`. The 19 tiles are exactly the 19 ccsc-tools repositories reproduced here. The CCDC page lists one dataset, FlareDB, which I cover separately in `reports/CCDC_Dataset_Analysis.md`.

On 2026-10-13 zenodo.org answered HTTP 504 (gateway time-out). I tried the API, the record pages and www.zenodo.org, and the Wayback Machine has no snapshots. I therefore resolved each DOI through DataCite and doi.org content negotiation, which return APA, BibTeX and the full metadata (`scripts/citations/fetch_ccsc_citations.py`). On 2026-10-14 Zenodo answered normally, and I then checked the six direct-upload archives (§3.5).

Several DOIs on the CCSC page differ from the badges in the GitHub READMEs. For example, the page gives 10.5281/zenodo.7506997 for FlareML while its README gives 10.5281/zenodo.5634114. I treated the page as authoritative.

3.2 Reproduction protocol

1. Read the README and notebook, and record the citation, data, weights and environment.

2. Make a working copy of the cited version (`git archive <tag>` for tagged releases, HEAD for direct uploads; see §3.5). The upstream clone is never modified.
3. Create an isolated environment with the closest available package versions (§4).
4. Run the released test or prediction entry point with the released weights and data, and log stdout.
5. Compare with the authors' stored outputs (result CSVs, logs, notebook outputs).
6. Compare with the paper's tables, on the paper's own test set where the release ships it, and otherwise with the closest comparable quantity, stating the difference.
7. Audit the evaluation: data provenance, train/test overlap, threshold and model selection, use of labels at inference, horizon definition and baselines.
8. Where the released evaluation was not out-of-sample, run an independent evaluation that changes only the evaluation and never the model. I label these as extensions, not reproductions.

3.3 What I changed and what I did not

Every result I report comes from a run I executed and has a log or JSON file behind it, and whenever I say "reproduced" I state whether I mean tool level or paper level. Compatibility edits are minimal and documented in each script's docstring, for example `Model(inputs=…)` for Keras 2, keyword arguments for scikit-learn ≥ 1.0, a `torch.load(weights_only)` shim for torch 1.12, a sorted `os.listdir`, replacing `K.tf` with `tf.compat.v1` for FibrilNet, and early-bound Lambda closures for SDNN (§5.4). Third-party artefacts (the FibrilNet and SDNN models and input from web.njit.edu, and the SolarCNN model from the README's Google Drive link) were downloaded only from the authors' own links and are not redistributed; their SHA-256 hashes are in `results/downloaded_artefacts_sha256.txt`. I did not modify any upstream repository or use any paid service.

3.4 Metrics

For classification I report recall, precision, ACC, BACC, HSS and TSS, plus AUC. TSS is insensitive to the class ratio and is the standard metric for flare forecasting. For regression I use RMSE, MAE, R^2, Pearson r and skill relative to persistence, and for images SSIM, PSNR, PCC, IoU and RMSE (in G). The definitions are in `scripts/common_metrics.py`.

3.5 Cited release vs the GitHub HEAD

For the 13 tagged tools I compared the cited tag with HEAD using `git diff --stat <tag> HEAD` (output in `results/cited_versions/cited_tag_vs_HEAD_diffstat.txt`). For SolarUnet, FlareML, RNN-CME, SEP, Dst, TSInet, Kp, CMETNet, FibrilNet and SDNN only documentation or environment files changed, so the HEAD results apply to the cited version unchanged. FibrilNet, SDNN and RNN-CME were in any case run from the cited tag. In SolarFlareNet v1.3 the difference is a 4-line change in `utils.py` that changes neither the features fed to the models nor the models.

LSTM-flare v1.0.2 is the exception. Its cited release ships older demo models than HEAD, and with those models the stored demo outputs are reproduced.

Class	Agreement with stored output	TSS (mine)	TSS (stored)
≥C	99.5 %	0.471	0.459
≥M	99.1 %	0.576	0.586
≥M5.0	99.5 %	0.707	0.700

The mismatch I reported earlier at HEAD therefore came from HEAD's newer models. For ≥M5.0 I took softmax column 1 as the positive class. The released test script still reads column 0, so as released the ≥M5.0 result is inverted (TSS −0.679).

SYMHnet v1.0 has 11 input features (HEAD has 8) and different weights, but the same data and the same next-row target. I repeated all 28 runs at v1.0.

For the six direct Zenodo uploads I read each zip's central directory with HTTP range requests and compared every entry's size and CRC-32 with the GitHub file at the same path (`scripts/citations/compare_zenodo_archives.py`).

Tool	Archive check
DeepHMI, SINet	Identical content; the files that differ do so only in line endings (CRLF vs LF)
LLMFlare	Identical (23 files)
DeepGeoCME	Identical in substance (HEAD's README adds one sentence linking to Zenodo). The archived model and test tensors have the same size and CRC-32 as the GitHub release assets I ran
MIST	Model and data identical. The archived `test.py` calls `os.path.join` without `import os`, so the cited version fails with `NameError`; HEAD adds the import. The archive also contains `data/processed/X_train.npy`, which GitHub lacks (used in §5.1)
SolarCNN	Restricted-access record with no public files, so it cannot be checked

For these tools the HEAD results therefore stand for the cited versions, apart from MIST's one-line fix.

---

4. Experimental Setup

All runs used a cloud container with 4 vCPUs, 15 GB of RAM, no GPU and about 30 GB of writable disk, on Linux 6.18.

At first only GitHub and PyPI were reachable. After I enabled full network access, nature.njit.edu, web.njit.edu, arXiv, nature.com, frontiersin.org and Google Drive were reachable too. Zenodo returned HTTP 504 on 2026-10-13 and was reachable on 2026-10-14, although SolarCNN's record is restricted-access. The AGU/Wiley, A&A and Springer full texts stayed unreachable (403 or paywall).

I used four environments, frozen in `tools/environments/`.

Environment	Main packages	Used for
tf210	Python 3.9.25, TensorFlow 2.10.1 (CPU), Keras 2.10, TFP 0.17, tensorflow-addons 0.19, scikit-learn 1.2.1, xgboost 1.5	The TensorFlow tools (the papers used TF 1.11–2.11)
sk024	Python 3.9, scikit-learn 0.24.2, sklearn-extensions 0.0.2	FlareML, whose pickles were written by scikit-learn 0.23.2
torch112	Python 3.9, PyTorch 1.12.1 (CPU), timm 1.0.9	MIST, SINet, DeepHMI and DeepGeoCME (MIST requests torch 2.3 and DeepGeoCME ≥2.6)
np2	Python 3.11, NumPy 2.0	Only reading MIST's scaler pickle

The upstream commit hashes are in `tools/upstream_commits.csv`, and the cited tags are in §2. Each tool's scripts are in `scripts/<tool>/` with usage notes in their docstrings. They are run from a working copy of the upstream repository, and the logs are in the matching `results/<tool>/` folder.

---

5. Reproduction Results

5.1 Flare forecasting

LSTM-flare-prediction

For the cited v1.0.2 of Liu et al. (2019), the released demo models reproduce the stored outputs with 99.1–99.5 % agreement (§3.5). For ≥M5.0, however, the test script reads softmax column 0 as the positive class, while the label encoder orders the classes ['N','X']. The ≥M5.0 predictions are therefore inverted, and the model as released scores TSS −0.679.

I then retrained the models on the split of the paper's Table 2, whose sample counts match the shipped `Table2_dataset` exactly. I used the released code with its hyper-parameters unchanged (7 epochs, batch 256, balanced class weights, and the tool's feature subsets of 20, 22 and 14 features) and ran 10 seeds for each class. The test period runs from 2015-01 to 2018-05. The train, validation and test HARPs are disjoint (0 shared ARs) and in chronological order.

Class (test n, positives)	Threshold	Recall	Precision	BACC	HSS	TSS
≥M5.0 (44,689, 180)	paper's 0.50	0.886	0.048	0.906	0.084	0.812 ± 0.119
	tool's fixed 0.75	0.778	0.068	0.867	0.117	0.734 ± 0.164
	chosen on 2014 validation	0.904	0.048	0.913	0.083	0.825 ± 0.104
	chosen on test	0.964	0.047	0.939	0.082	0.877 ± 0.035
≥M (44,689, 1,278)	paper's and tool's 0.60	0.810	0.233	0.865	0.331	0.730 ± 0.031
	chosen on validation	0.864	0.173	0.869	0.251	0.739 ± 0.022
	chosen on test	0.846	0.206	0.874	0.298	0.748 ± 0.021
≥C (44,689, 8,732)	paper's and tool's 0.50	0.722	0.555	0.790	0.521	0.580 ± 0.011
	chosen on validation	0.748	0.528	0.792	0.505	0.584 ± 0.009
	chosen on test	0.757	0.524	0.795	0.504	0.589 ± 0.009

The paper's Tables 3 and 4 give TSS 0.877, 0.790 and 0.607 for ≥M5.0, ≥M and ≥C, using the LSTM with all 40 features and thresholds of 0.50, 0.60 and 0.50. At those thresholds my retrained models give 0.812, 0.730 and 0.580, which is 0.03–0.07 lower. The feature subsets are not the ones in Table 4, because the tool uses the best features from Table 5.

I also re-ran the protocol of the paper's Table 5 (≥C, 10×10 cross-validation) with the original `LSTMpredict.py`, after a 4-line patch for TF2. This meant 100 trainings and took 86 CPU-minutes.

Metric	Paper Table 5 and authors' `output.txt` (TF 1.12)	Re-run (TF 2.10)
Recall	0.773 (0.027)	0.741 (0.023)
Precision	0.541 (0.030)	0.531 (0.024)
ACC	0.826 (0.015)	0.821 (0.012)
BACC	0.806 (0.004)	0.790 (0.005)
HSS	0.526 (0.021)	0.505 (0.018)
TSS	0.612 (0.009)	0.581 (0.011)

The authors' `output.txt` matches the ≥C column of the paper's Table 5 digit for digit. My re-run is 0.031 lower in TSS. Switching the recurrent activation to the `hard_sigmoid` of Keras 2.2.4 did not change this, so I attribute the gap to differences between the frameworks.

In the code, the history-masking check tests `row` instead of `prev_row`, so rows with incomplete history are never masked. The Table 5 threshold is the one that maximises the mean TSS on the test folds.

MIST

The cited version is Zenodo record 16780646. It is identical to the GitHub HEAD except that the archived `test.py` lacks `import os` and fails with a `NameError`, so I ran HEAD. The released `test.py` gives TSS 0.9062 (TP 493, FN 49, FP 56, TN 16,313), which matches the 0.90 in the README.

The title of the software record ("A New Deep Learning Method for Solar Flare Prediction") does not match any paper I could find. The closest is arXiv:2606.21896 (Chaudhary, Wang, Xu, Xu & Zhang 2026). It describes the same design, with 16 SHARP features, T = 24, patch length 2, d = 128, data from May 2010 to May 2018, and a stratified random train/test split with Optuna thresholds tuned "to maximize the true skill statistic on the test set". Its reported test numbers differ from those of the released model on the released split, so the two are related but not the same experiment.

	Paper (arXiv:2606.21896)	Released model and split
Train / test windows	67,661 / 16,916 (553 positives)	67,642 / 16,911 (542 positives)
Accuracy	95.18 %	99.38 %
Recall	0.89	0.910
AUC	0.96	0.997

The file `processed_solar_dataset.csv` matches the LSTM Table 2 ≥M training file (2010-05 to 2014-01) to within 4.4×10^{-16}, and the labels are identical. Each window is rows p to p+23 and is labelled by row p+24. The 80/20 split is random over windows, and the scaler is fitted on all 84,577 rows.

When I reconstructed the windows from the CSV, 16,172 of the 16,911 test windows (95.6 %) had an adjacent window, shifted by one hour, in the training set. The Zenodo archive also contains the authors' own training array, `X_train.npy` (67,642 windows, matching `y_train`), and it confirms the overlap. Compared exactly with that array, 16,195 test windows (95.8 %) share 23 of their 24 rows with a training window, and 132 are identical to one. These include 522 of the 542 positive test windows (96.3 %). The small excess over the reconstruction comes from windows whose rows also recur elsewhere in the data.

As an out-of-sample check I scored the same weights, with the same preprocessing, on the 2014 and 2015–2018 files.

Evaluation set	Windows	Positives	AUC	TSS at 0.5
Released random-split test	16,911	542	0.997	0.906
2014 (validation file)	26,449	1,347	0.835	0.208
2015–2018 (test file)	44,665	1,278	0.833	0.197

With the threshold chosen on 2014 instead of 0.5, the TSS on 2015–2018 is 0.364.

SolarFlareNet

At the cited tag v1.3, the 9 shipped checkpoints load completely, and the released tests reproduce the stored result CSVs with at least 99.98 % agreement for 8 of the 9. The paper reports 10-fold cross-validated TSS of 0.818, 0.839 and 0.835 at 24 h (≥M5.0, ≥M and ≥C), and 0.71–0.74 at 48 and 72 h.

On the bundled 2022 test files, 8 of the 9 shipped models have no skill (AUC 0.40–0.53, TSS −0.04 to 0.05). This is true both in my run and in the authors' stored files. The ninth model (C/24 h) gives TSS 0.792, but 84 % of the 24 h test rows (same HARP and timestamp) also appear in `training_data_*_24.csv`. On the 2,581 rows that are not in training, its TSS is 0.697.

The loader selects features by position, using the column order of another dataset, and its `start_feature=5` skips the first feature. The 9 features it feeds are therefore TOTPOT, ABSNJZH, USFLUX, AREA_ACR, MEANPOT, R_VALUE, MEANGAM, MEANJZH and MEANJZD, not the paper's 9, which include TOTUSJH, TOTUSJZ, MEANALP and SAVNCPP.

FlareML

With the cited v1.5, the released pretrained ensemble reproduces the shipped log on the 40-sample demo, with accuracy 0.850 for ENS, 0.825 for RF, 0.725 for ELM and 0.300 for MLP. Of the 40 demo test rows, 31 match training rows on all 13 features.

The code also adjusts what it displays in two places. `check_pm_precision` replaces RF's displayed score by ENS − U(0.01, 0.07) whenever RF ≥ ENS, and `log_cv_report` edits the FP and FN cells when they are 0.

The paper's Table III gives an average TSS of 0.507 for ENS and 0.488 for RF. As an extension, I evaluated RF on the 845-flare table with the shipped hyper-parameters and 5 seeds.

Protocol	Macro-TSS
Random 10-fold	0.338 ± 0.007
AR-grouped 10-fold	0.246 ± 0.022
Chronological	0.363 ± 0.009

The paper's exact protocol is not in the release, so I cannot attribute the gap to 0.488.

5.2 CME and SEP

RNN-CME-prediction

I ran the cited v1.3 from its tag. On the 2-AR demo, the predicted labels of all 10 models (GRU and LSTM, 12–60 h) are identical to the stored outputs, with TSS 0.817–1.000.

The folder `CME_data_samples/` holds the paper's full test sets, and their row counts equal its Table 1 (for example 922 + 1,059 at 24 h). I scored the released models on these sets.

Model	n (pos)	AUC	TSS, tool threshold	TSS, best threshold	Paper Table 4 TSS / AUC
GRU 12 h	1,312 (550)	0.753	0.310	0.414	0.452 / 0.779
GRU 24 h	1,981 (922)	0.767	0.382	0.452	0.588 / 0.850
GRU 36 h	2,467 (1,214)	0.741	0.486	0.518	0.602 / 0.874
GRU 48 h	2,900 (1,540)	0.719	0.351	0.372	0.588 / 0.822
GRU 60 h	3,297 (1,814)	0.731	0.471	0.480	0.600 / 0.870
LSTM 12–60 h	as above	0.635–0.786	0.263–0.372	0.369–0.426	0.459–0.562 / 0.791–0.858

The full test files contain no 'padding' rows. On the two demo ARs, building the history without them changes 0–4.4 % of the predicted labels, so this does not explain the gap. The paper's numbers are also means over 10 CV runs, whereas mine come from the single released models.

The released data are also inconsistent. The 18 feature columns in `CMEpredict/normalized_*.csv`, which were used to train the released models, hold the same values as `CME_data_samples/` but in a different order. Under the `CME_data_samples` header, the extensive parameters correlate with USFLUX as expected (TOTUSJH 0.92, TOTPOT 0.89, AREA_ACR 0.92). Under the order assumed by `CMEpredict.py` they do not (TOTPOT 0.06, AREA_ACR −0.11), while "MEANGBT" correlates at 0.92. The feature-index lists in the code reproduce the ranking in the paper's Table 2 exactly under this second naming. For example, the GRU 24 h model's "MEANPOT, SHRGT45, MEANJZH, …" are ABSNJZH, TOTUSJZ, MEANJZD, … under the physically consistent naming. Either the column order of the training files or the feature names behind the paper's ranking are mislabelled, and the release does not show which.

The quality filters in `load_data` and `output_result` index columns −5/−4/−2/−1 for radial velocity, QUALITY and longitude. Those columns do not exist in the 22-column files, so the filters compare normalised SHARP values with 3,500, 65,536 and 70 and never trigger.

SEP-prediction

All 12 stored metric rows of the cited v1.4 (FC_S and F_S, 12–72 h) are reproduced exactly. For example, FC_S at 24 h gives TSS 0.9782. The test period is April to September 2017 and contains 3 ARs, and all positives come from 2 of them (12665 and 12673). The calibrated Brier skill uses an isotonic regression fitted on the test set.

The paper's Tables 3–4 report cross-validation means, for example a biLSTM TSS for FC_S of 0.737, 0.515, 0.567, 0.592, 0.590 and 0.703 (12–72 h), and 0.441–0.529 for F_S. The release ships no CV split or code, and its sample counts differ from the paper's Table 1 (for FC_S 12 h, 4,043 in the release against 3,946 in the paper), so the paper's numbers cannot be re-run. The stored demo TSS (0.86–0.99 on 3 ARs) is not comparable with them.

CMETNet

For the cited v.1.1 I ran only the numerical COMB ensemble, not the CNN. The released script trains each regressor 10 times on random 200-event subsamples and keeps the round with the lowest MAE on the hold-out year. I re-implemented this loop for every year with at least 3 events, using the same estimators and hyper-parameters (18 hold-out years, 348 events), and compared different ways of choosing among the 10 rounds.

Choice of round	Event-weighted MAE over 18 years (h)	Mean over 2013–2017 (h)
Best on the test year (released)	12.36	14.2
Best on the previous year (validation)	13.09	15.3
None (average over rounds)	13.41	15.6

Selection on the test year beats the average round in 18 of 18 years, with an optimism of 0.3–2.9 h. The paper uses leave-one-year-out over 2013–2021 (9 runs) and reports an MAE of 10.32 h for the numerical-only FW, and 9.75 h with r = 0.83 for CMETNet with the CNN. The released data have fewer than 3 events per year for 2018–2021, so only 2013–2017 overlaps with the paper, and over those years COMB reaches 14.2 h even with test-year selection. The script `pretrained_models_results.py` plots hard-coded MAE and PPMCC values rather than computing them.

DeepGeoCME

The cited version is Zenodo record 22955090, which is identical in substance to the GitHub HEAD. Its model and test tensors match the GitHub release assets, checked by SHA-256. My confusion matrix on the 33 test events is identical to the shipped figure (TN 14, FP 2, FN 2, TP 15), giving TSS 0.757, recall 0.882, precision 0.882 and Brier score 0.080. This equals the paper's headline TSS of 0.757 from its 80:20 split. The paper's 5-fold CV result (TSS 0.703 ± 0.030) needs the training images, which are not shipped.

5.3 Geomagnetic indices, radio flux, irradiance

Dst-prediction (DSTT)

In the cited v1.1, `DSTTModel.predict()` passes each prediction p and its true value a through `t(a, p, 0.9)`. The prediction is kept only if it equals a or differs from it by at least 90 %. Otherwise it is replaced by $0.9 \cdot a$. The stored predictions therefore equal 0.9 × label exactly for 88–90 % of hours. `DSTT_test.test()` also loads the 6 h weights for every horizon, and the loader's test-exclusion filter removes 0 rows.

To evaluate the model without the label substitution, I took the mean of 50 MC passes, using the correct weights for each horizon, on the paper's test period of October–November 2021. This period has 1,103 hours, where the paper states 1,104. Each cell below gives R^2 with RMSE in nT in brackets. The paper's values are read off the bars of its Fig. 3 and are approximate.

h	Released path	Raw model	Persistence	Paper Fig. 3 (DSTT)
1	0.942 (3.97)	0.063 (15.98)	0.934 (4.24)	0.75 (5.5)
2	0.942 (3.97)	0.076 (15.88)	0.836 (6.70)	0.69 (5.2)
3	0.938 (4.11)	0.340 (13.43)	0.750 (8.27)	0.72 (6.0)
4	0.940 (4.05)	0.380 (13.02)	0.674 (9.45)	0.66 (5.8)
5	0.938 (4.11)	0.127 (15.46)	0.599 (10.47)	0.67 (5.5)
6	0.932 (4.30)	0.338 (13.47)	0.525 (11.41)	0.59 (5.7)

The paper reports these results only as figures, and its RMSE and R^2 values are not consistent with each other for this test period. The standard deviation of Dst over these hours is 16.5 nT, so an RMSE of about 5.5 nT implies an R^2 of about 0.89, while an R^2 of about 0.75 implies an RMSE of about 8.3 nT. The raw model is far below the paper. The released path scores above it because of the label substitution.

Kp-prediction (KpNet)

At the cited v1.1, my re-run metrics match the stored `default_results` within the stochastic variation of the variational layers. For example, the 1 h R^2 is 0.629 against 0.625 stored, and r = 0.801 in both. The model is trained on 2010–2021 and tested on January–March 2022, with no shared timestamps. Persistence does better at 1–2 h (R^2 0.82 and 0.65, against 0.63 and 0.49 for KpNet), while KpNet is better from 3 h onward (at 6 h, 0.26 against 0.08). The paper (ISMIS, Springer) was not accessible to me.

SYMHnet

I re-ran the cited tag v1.0 and also ran HEAD. At both versions the released data folders for 1 h and 2 h are byte-identical, and the test windowing targets the next row (+1 min or +5 min). Measured against

one-step persistence, the skill score SS = 1 − MSE/MSE_ref is negative in all 28 runs at both versions, ranging from −1.3 to −39.8 at v1.0. Against h-hour persistence it is positive, from +0.44 to +0.99 at v1.0.

Below, the paper's Tables 8, 9 and 12 (5-min data, SYMHnet column) are set against my re-run of the cited v1.0 code.

Storm	Paper RMSE (1 h / 2 h)	v1.0 RMSE (1 h / 2 h)	1-step persistence RMSE	R^2 at 1 h (paper / v1.0)
31	7.923 / 9.737	7.868 / 9.547	2.33	0.976 / 0.976
33	11.366 / 13.745	11.512 / 13.829	4.74	0.979 / 0.978
42	3.900 / 4.674	3.831 / 4.739	1.19	0.985 / 0.985
37	10.320 / 10.923	11.699 / 14.092	3.56	0.993 / 0.980
28	4.599 / 4.977	2.421 / 2.663	1.48	0.957 / 0.988
36	5.618 / 5.970	2.875 / 3.071	1.90	0.949 / 0.987
40	5.752 / 6.336	2.908 / 3.469	1.40	0.948 / 0.987

For 3 of the 7 storms (31, 33 and 42) the cited code reproduces the paper's RMSE and R^2 within about 2 %. For storm 37 my RMSE is higher, and for the other three it is about half the paper's value. Since this code scores a next-row target, the agreement suggests that the paper's "1-h / 2-h ahead" numbers come from the same pipeline. If so, simply repeating the last 5-min value would have given a lower RMSE than SYMHnet on every storm in the table. The paper's FSS uses the Burton equation as its reference, which I did not re-implement. The newer models at HEAD give RMSE of 1.8–5.6 nT on the same storms, which is further from the paper.

SINet

The cited version is Zenodo record 20274899, which is identical in content to GitHub HEAD, so I ran HEAD. The table gives RMSE at horizons of 1, 27, 45 and 60 days (in sfu for F10.7).

Series	1 d	27 d	45 d	60 d
F10.7, SINet re-run	3.8801	13.7411	15.0921	16.3241
F10.7, best naive baseline	4.26	15.31	16.94	18.66
F30, SINet re-run	2.0466	7.7862	9.1215	10.1475
F30, authors' notebook	2.0586	7.6491	9.1584	9.9921
F30, best naive baseline	2.27	7.96	9.89	11.38

My F10.7 values are identical to the notebook to 4 decimals, and the paper (arXiv:2604.10045) reports 3.88 at 1 d and 16.32 at 60 d, so this result is reproduced. My F30 values are within 2 % of the notebook's, and the paper reports 9.99 at 60 d. As an extension, I scored persistence and 27-day recurrence on the same windows and took the better of the two as the baseline. SINet beats both baselines at every horizon, by 9–13 % for F10.7 and 2–11 % for F30.

TSInet

In the cited v1.6, `process_args()` lacks a `global` declaration, so `-n NRLTSI2/SATIRE-S` silently reconstructs TCTE instead. I worked around this with a three-line driver that sets the module globals. The 30-day default reconstructions give RMSE of 0.71–0.79 W m^{-2}, against 0.62–0.72 in the authors' stored files, because of the stochastic periodic re-fitting. A naive baseline that repeats the last seed value gives 0.09–0.23 W m^{-2} on the same windows. The paper reports its results only in figures (Figs. 6–10).

5.4 Image-based tools

SolarUnet

The cited v.1.2 pipeline iterates over `os.listdir()` without sorting. On Linux the order was frame_1, frame_3, frame_2, which pairs masks with the wrong FITS frames. With a sorted listing, the 3 predicted masks are byte-identical to the authors', and the post-processed and tracking images agree on at least 99.99 % of pixels. The statistics in the paper's Table 2 use 147 test magnetograms, of which 3 are shipped.

FibrilNet

I ran the cited v1.6 from the tag, with the model from web.njit.edu. The released `test.py` cannot run as shipped. It calls `predict(model, image, T=2)`, but no function of that name is defined anywhere in the release, and `helper.predict` has a different signature. I therefore ran the notebook path instead, using `helper.predict` with MC dropout at T = 5 under TF 2.10. The only shims I added supply the `K.tf` and `np` names that Keras 2.2.4 used to expose.

MC dropout makes the masks stochastic, so I ran 3 seeds of about 2.5 CPU-minutes each and compared them with each other and with the authors' stored masks (`results/FibrilNet/compare_runs.json`). The IoU range is given in brackets.

Comparison (15 pairs each)	Pixel agreement	Fibril IoU	Dice
Ours vs authors' stored masks	0.991	0.901 (0.885–0.911)	0.948
Ours vs ours, different seeds	0.991	0.900 (0.886–0.911)	0.948

The stored masks are statistically indistinguishable from another MC draw of the same model, so I consider FibrilNet reproduced. Checking the paper's Table 1 (PS of at least 95 %, FCFP of about 80 % and RI of at least 91 %, all against the Jing et al. 2011 tool) would need that tool's skeletons for the 5 test images, which are not shipped.

SDNN

I ran the cited v1.3 from the tag, with the model and input from web.njit.edu. Under Keras 2.10 the released `SDNN()` fails at `predict()` with "TypeError: Could not build a TypeSpec for KerasTensor". Each branch has two lambdas, `Lambda(lambda x: x[:, :, i])` and `Lambda(lambda x: K.expand_dims(out, -1))`, which close over loop variables. TF 1.x evaluated them once at graph build, but Keras 2.10 executes them again.

I rebuilt the network with the same layers, binding the two lambdas as TF 1.x had evaluated them, and copied the Keras 2.2.4 weights layer by layer by name (35 weighted layers, with shapes asserted). The authors' `read_data`, `inverse` and `save_results` then ran unchanged. Reading the data took 103 s, and the inversion of 518,400 pixels took 873 s on 2 threads.

All 5 output maps match the stored ones, with Pearson r = 1.000000 for every map (`compare_vs_author.json`).

Map	Max |Δ|	Reference std
Bx	0.011 G	373 G
By	0.0099 G	251 G
Bz	0.0032 G	554 G
Doppler width	1.4×10^{-6}	0.071
LOS velocity	3.5×10^{-6} km/s	0.84 km/s

The shipped input is the paper's test image D3 (2017-07-13 16:20:12, AR 12665). The paper's Table 1 reports a PPMCC of 0.912 for LOS velocity and 0.716 for Doppler width against Milne–Eddington (ME) inversion. Checking these would need the ME maps, which are not shipped.

SolarCNN

The cited version is Zenodo record 15250172, which is restricted-access, so I took the model from the Google Drive link in the README. The Drive archive's `saved_model.pb` and `variables.index` are byte-identical to the repository's, and it adds the missing `variables.data` (185 MB). With it, the released `test.py` reproduces the 3 shipped predictions to within 3×10^{-5} G (max |Δ|).

However, `test.py` feeds in raw Gauss values, whereas `train.py` trains on clip(±2000 G)/2000. I therefore also scored the model in the training space with the paper's metrics, taking the mean over the 3 shipped pairs (`eval_normalised.json`).

Setting	SSIM	PCC	PSNR (dB)
MDI input itself (the paper's "None")	0.812	0.943	32.63
Released `test.py`, raw input	0.806	0.943	29.35
Normalised input, as trained	0.892	0.954	36.77
Paper Table 3, SolarCNN vs None (76 test samples)	0.904 vs 0.833	0.884 vs 0.863	37.40 vs 34.73

With the training normalisation, the model improves on its input by about as much as the paper reports, but the released test path hides this. The paper's 76-sample test set is in the restricted-access Zenodo record.

DeepHMI

The cited version is Zenodo record 17059248, which is identical in content to GitHub HEAD. One U-Net pass takes about 67 s on this CPU, so a full run of 1,000 steps × 2 components would take about 37 h, and I was blocked by compute. Instead, I re-scored the authors' shipped output on the single test pair against GST/NIRIS, alongside the HMI input.

Component	SSIM (output / HMI)	RMSE in G (output / HMI)	PCC (output / HMI)
Bz	0.810 / 0.733	190 / 220	0.972 / 0.965
Bt	0.659 / 0.683	206 / 223	0.896 / 0.906

The A&A full text was not accessible (403).

MagNet

MagNet is listed on SolarDB at nature.njit.edu/solardb/magnet as "Un-published code". The package lists code, a 1.18 GB weights file (Keras 2.6, weights only), and 12 MDI and 12 HMI validation cases with the authors' predicted Bx/By. I read the index of the 1.08 GB `MagNet.zip` and the attributes of the weights file with HTTP range requests (`scripts/MagNet/check_package.py`).

`MagNet_model.py` does `from layers.attention import PAM, CAM`, but no `layers/` module exists in the zip or in the folder listing. The weights hold only a scalar `gamma` for each PAM/CAM layer (PAM −0.0078 and 0.0043; CAM 0.437 and −0.586), which is consistent with a DANet-style attention module that is not shipped. Running MagNet would need a third-party PAM/CAM implementation, and the result would then not be a reproduction of the released code, so I stopped there.

---

6. Comparison with Reported Results

Tool	Paper	Release as shipped	My result	Verdict
LSTM-flare	Tables 3 and 4, single split: TSS 0.877 / 0.790 / 0.607 (≥M5.0 / ≥M / ≥C); Table 5 CV: 0.881 / 0.792 / 0.612	demo; ≥M5.0 inverted	retrained at paper thresholds: 0.812 / 0.730 / 0.580; Table 5 ≥C CV 0.581	close, 0.03–0.07 lower

Tool	Paper	Release as shipped	My result	Verdict
MIST	arXiv:2606.21896, random stratified split: ACC 95.18 %, recall 0.89, AUC 0.96	README: TSS 0.906	ACC 99.38 %, recall 0.910, AUC 0.997; on 2015–18, TSS 0.197	matches README exactly, not paper; not robust out of sample
SolarFlareNet	24 h, 10-fold CV: TSS 0.818 / 0.839 / 0.835	stored CSVs	8 of 9 models TSS about 0; C/24 h 0.792 (0.697 without train overlap)	not reproduced
FlareML	Table III, average TSS: ENS 0.507, RF 0.488	40-row demo log	RF 10-fold: 0.338 (random), 0.246 (AR-grouped)	lower; paper protocol unclear
RNN-CME	Table 4, GRU: TSS 0.45–0.60, AUC 0.78–0.87	2-AR demo, exact	paper test set: GRU 0.31–0.49 (AUC 0.72–0.77), LSTM 0.26–0.37	not reproduced; column order inconsistent
SEP	Tables 3 and 4, biLSTM CV: TSS 0.43–0.74	3-AR demo, TSS 0.86–0.99	demo exact	paper not re-runnable from release
CMETNet	MAE 9.75 h, r 0.83; FW 10.32 h (2013–21)	hard-coded plot	COMB 12.36 h (18 years); 14.2 h (2013–17)	not reached, even with the best model chosen on the test year
DeepGeoCME	TSS 0.757 (80:20 split); CV 0.703	confusion matrix	identical, TSS 0.757	paper headline reproduced
Dst	Fig. 3: R^2 about 0.59–0.75, RMSE about 5–6 nT	R^2 about 0.94 via label substitution	raw model R^2 0.06–0.38	not reproduced
Kp	paper not accessible	stored results	within MC noise	tool level
SYMHnet	Tables 8, 9 and 12: per-storm RMSE and R^2 (5-min)	target is the next row	v1.0: storms 31/33/42 within about 2 %; 28/36/40 about half; 37 higher	partly; horizon definition questioned
SINet	F10.7 RMSE 3.88 (1 d), 16.32 (60 d); F30 9.99 (60 d)	notebook	3.8801, 16.3241; F30 10.1475	reproduced
TSInet	figures only	stored 30-day runs	similar RMSE; naive baseline better	tool level
SolarUnet	Table 2 statistics on 147 frames	3 demo frames	byte-identical	tool level
FibrilNet	Table 1 vs Jing et al.: PS ≥ 95 %, FCFP about 80 %, RI ≥ 91 %	stored masks	IoU with stored masks equals IoU between my own seeds (0.90)	tool level; reference not shipped
SDNN	Table 1 (D3) vs ME: PPMCC 0.912 (LOS v), 0.716 (Doppler w.)	stored maps	r = 1.000000 with stored maps	tool level; ME reference not shipped
SolarCNN	Table 3: SSIM 0.904 vs 0.833 (None), PSNR 37.40 vs 34.73	shipped predictions, no gain	exact to 3e-5 G; after normalisation SSIM 0.892 vs 0.812, PSNR 36.8 vs 32.6	partly; same direction and size, but on 3 of 76 samples
DeepHMI	A&A paper not accessible	1 output	re-scored only	blocked by compute

7. Limitations and Challenges

1. Zenodo was unreachable on the first day (HTTP 504), so I compared the six direct-upload archives with GitHub on the second day (§3.5). SolarCNN's record, which holds its full 76-sample test set, is restricted-access, so I could not check that archive or run the paper-level test.

2. I read the full texts on arXiv, *Scientific Reports* and *Frontiers*. The AGU/Wiley version of SINet (JGR), the A&A paper for DeepHMI and the Springer paper for KpNet returned 403 or a paywall. For SINet I used the arXiv version instead. Values I read off bar charts (Dst) are approximate.
3. Several papers report cross-validation means (SEP, RNN-CME, SolarFlareNet, LSTM Table 5) or compare against reference products (FibrilNet, SDNN) whose data or code are not in the releases. My paper-level comparison is limited to what each release supports.
4. The tools were written for TensorFlow 1.11–2.11 and PyTorch 1.12–2.6, and I ran TensorFlow 2.10 and PyTorch 1.12 on CPU. Most outputs were exact, which indicates numerical stability across versions. The LSTM Table 5 gap (−0.03 TSS) may be version-related.
5. Several models are stochastic and cannot be reproduced bit for bit. These are the variational or MC-dropout layers in Dst, Kp, SYMHnet and FibrilNet, the diffusion model in DeepHMI, random subsampling in CMETNet and re-fitting in TSInet. For these I compare metrics, and for FibrilNet I also compare against run-to-run variation.
6. I had no GPU. DeepHMI's sampler (about 37 CPU-hours), the cross-validation retraining for SolarFlareNet and SEP, and FibrilNet training were therefore out of scope.
7. The demos come with small evaluation sets (1–3 images, 2–3 ARs for SEP and RNN-CME, 33 events for DeepGeoCME, 3 SolarCNN pairs), so many of the numbers I reproduced are statistically fragile.
8. My findings concern the released software at the cited versions. Where a release and its paper differ, the original experimental code may explain the difference, and I have not concluded otherwise.

---

8. Understanding of Each Tool

FlareML is a tabular 4-class classifier. Thirteen SHARP parameters taken at the start of a flare day feed an RF, an MLP and an ELM (an ELM has a random hidden layer and a closed-form output layer), and their outputs are combined by majority vote. Skill is reported as one-vs-rest TSS for each class.

LSTM-flare is a sequence classifier. The last 10 hourly vectors of SHARP parameters and flare history for an AR go into an LSTM with temporal attention, which outputs the probability of a ≥C, ≥M or ≥M5.0 flare within 24 h. Weighted cross-entropy deals with the class imbalance, and a threshold turns the probability into an alert.

RNN-CME uses the same kind of model (GRU or LSTM with attention), but only once an M- or X-class flare has occurred. It predicts whether the eruption carries a CME, which is a key step towards predicting geoeffectiveness.

SolarFlareNet is a deeper, transformer-style sequence model (Conv1D, then LSTM, then 4 encoder blocks) for 24, 48 and 72 h horizons. Its probabilistic output is based on calibration, and the model is run operationally.

SEP-prediction applies a BiLSTM with attention to SHARP sequences to predict whether a flaring AR produces an SEP event. The output probabilities are calibrated with isotonic regression.

CMETNet predicts the Sun-to-Earth transit time of a CME. A regression ensemble (RF, SVR, XGBoost and GPR, combined by taking the median) works on CME and solar-wind features, and its output is fused with a CNN applied to LASCO C2 images.

DSTT, KpNet and SYMHnet forecast geomagnetic indices from solar-wind data. Convolutional, recurrent and attention feature extractors feed Bayesian output layers (TFP variational layers or MC dropout). Repeated stochastic passes give a mean prediction together with aleatoric and epistemic spread. SYMHnet also has a graph layer that treats the solar-wind parameters as nodes.

SINet follows the TimesNet design. An FFT finds the dominant periods (for example the 27-day solar rotation), the 1-D series is reshaped into 2-D by period, and Inception convolutions are applied to give multi-day F10.7 and F30 forecasts.

TSInet is an autoregressive Conv-LSTM-attention generator started from 7 observed TSI values. It rolls forward and periodically retrains itself on its own outputs.

SolarUnet and FibrilNet do encoder–decoder segmentation of BBSO/GST images. Both are trained on labels from classical threshold-based tools (SWAMIS, and the Jing et al. 2011 fibril tracer). SolarUnet is a U-Net with feature tracking. FibrilNet is a SegNet-style network that reuses the max-pooling indices for unpooling and adds skip concatenations. Its dropout stays active at test time, so T stochastic passes give a mean mask plus aleatoric and epistemic maps, and a polynomial fit gives the fibril orientations.

SDNN passes each Stokes profile (I, Q, U, V; 60 spectral points) through its own 1-D CNN branch. The four branches are stacked and go through a second CNN and dense layers, which output |B|, inclination, azimuth, Doppler width and LOS velocity for each pixel. One forward pass thus replaces iterative Milne–Eddington fitting.

SolarCNN, DeepHMI and MagNet translate images from one instrument to another, with the scientific aim of homogenising multi-decade archives across instruments. SolarCNN brings MDI data up to HMI quality with an attention CNN trained on clipped, scaled fields. DeepHMI raises HMI data to GST resolution with a conditional diffusion model. MagNet estimates the transverse field from MDI LOS and Hα data, using two ResNet-101/DANet heads trained with HMI vector data.

MIST is a patch-based time-series transformer. It cuts 24 h of 16 SHARP features into 2 h patches, embeds and encodes them, and outputs the probability of a ≥M flare.

LLMFlare adds explanations to MIST. SHAP attributions for MIST predictions are turned into narratives by a LoRA-tuned Llama-3.1-8B, and an LLM then judges the narratives.

DeepGeoCME is a multimodal image-fusion classifier (ResNet-50 and EfficientNet-B0 with attention) that predicts whether a CME will be geoeffective.

---

9. Potential Research Directions

1. The checks in §5 could be turned into a reusable leakage and evaluation test suite for heliophysics ML, run alongside every CCSC tool. It would check for AR or time overlap between training and test data, adjacent windows, thresholds or models chosen on the test set, access to labels at inference, inconsistent horizons and wrong column labels (the RNN-CME check takes seconds), and it would compare each model with a persistence baseline.
2. Each CCDC/SolarDB dataset could ship as AI-ready data with a split manifest of grouped chronological splits, a data dictionary (units, missing-value codes, column order), checksums and lineage. The undocumented fact that the MIST data is taken from the LSTM Table 2 training file, and the RNN-CME column orders, show why this is needed. It fits the aims of NSF 26-512.
3. My first e-mail asked about external, cross-instrument validation. For MagNet, SolarCNN and DeepHMI, error could be measured separately by instrument (MDI, HMI, IVM, GST), seeing and data quality, field strength and coverage. This would flag where reconstructed fields in the historical archive should be interpreted with caution. The SolarCNN normalisation finding shows that the inference path itself has to be part of such an audit.
4. I would also test whether the Bayesian intervals of DSTT, KpNet, SYMHnet and FibrilNet reach nominal coverage without ad-hoc scale factors, and report proper scoring rules.
5. For small test sets, bootstrap confidence intervals for TSS, resampled over ARs, would give event-level uncertainty and make results on 2–3 ARs interpretable.

---

10. References

The 19 tool publications and their software citations are listed in §2 and in `papers/citations/`. Other references:

- Bloomfield D. S., Higgins P. A., McAteer R. T. J., Gallagher P. T. 2012, *Toward Reliable Benchmarking of Solar Flare Forecasting Methods*, ApJL 747:L41.
- Jing J. et al. 2011, the threshold-based fibril tracing tool used as FibrilNet's reference (cited in Jiang et al. 2021).

- Nita G. et al. 2022, *Revisiting the Solar Research Cyberinfrastructure Needs*, arXiv:2203.09544.
- Liu N. et al. 2026, *FlareDB*, Scientific Data, doi:10.1038/s41597-026-06607-7; dataset Zenodo doi:10.5281/zenodo.16790538.
- Poduval B. et al. 2023, *AI-ready data in space science and solar physics*, Front. Astron. Space Sci. 10, doi:10.3389/fspas.2023.1203598.
- NSF 26-512, *Unlocking Dataset Value for AI-Enabled Scientific Discovery (AI Datasets)*.
- NASA ROSES-2022 B.16 *Heliophysics AI/ML-Ready Data* (NNH22ZDA001N-HARD).